

HOMEWORK_03

→ Objectives

- To practice creating, managing, and terminating processes using system calls in C.
- To manipulate files and directories with system calls.
- To demonstrate error handling in system-level programming.

→ Description

In this homework, you will implement a program that performs file manipulation and manages multiple child processes to perform tasks on files in a given directory. The program will:

- ☒ Accept a directory as a command-line argument.
- ☒ For each file in the directory ...
 - create a child process that prints the file name, its size, and the number of words in the file.
- ☒ Ensure the parent process waits for all child processes to complete before exiting.
- ☒ Implement basic error handling:
 - check for invalid directories, failed process creation, and inaccessible files.

Detailed Requirements:

- **Command-line Arguments:**
 - I) The program should accept one command-line argument, which is the directory name.
 - II) If no argument is provided, the program should print an error message and exit.
 - III) If the directory is invalid or doesn't exist, print an error message and exit.
- **File and Directory Traversal:**
 - I) Traverse the directory and list all files (excluding subdirectories)
 - II) For each file, create a child process using fork().
 - III) The child process will:
 - 1) Print the file name.
 - 2) Print the file size (in bytes)
 - 3) Count and print the number of words in the file (assume words are separated by spaces).***only for the .txt files

- 4) Use stat() or lstat() to get the file size.
- 5) You may use standard I/O functions like fopen(), fscanf(), or system calls like read() for word counting.

- **Process Management:**

- I) Ensure that the parent process waits for all child processes to complete using wait() or waitpid().
- II) Handle any failed fork() calls by printing an error message and terminating the program.

- **Error Handling:**

- I) Handle cases where the file cannot be opened or read (e.g., permissions issues) and print appropriate error messages.
- II) Make sure the program can handle edge cases such as empty files, very large files, or directories with no files.

- **Extra:**

- I) In addition to printing file names, file sizes, and word counts, the program use additional command-line options to filter and display files based on their ownership.

→ **Command-line options:**

-u <username>: List only the files owned by the specified user.

Program Documentation and Testing

- 1) Use appropriate names for variables and functions.
- 2) Use a Makefile to compile your program.
- 3) Include meaningful comments to indicate various operations performed by the program.
- 4) Program must include the following header information within comments:

```
/*  
Name:  
BlazerId:  
Project #:  
To compile: <instructions for compiling the program>  
To run: <instructions to run the program>  
*/
```

- 5) Test your program with the sample test cases provided as well as your own test cases.
- 6) You can include any comments you have about testing in the README.txt

→ Examples:

Command	Description
<code>./hw3 /path/to/directory</code>	File: file1.txt Size: 1234 bytes Words: 200 File: file2.txt Size: 5678 bytes Words: 500 File: file3.txt Size: 345 bytes Words: 50 ...
If the directory does not exist:	Error: Directory not found.
If no argument is provided:	Usages: <code>./hw3 <directory_name></code>
<code>./hw3 -u <username> /path/to/directory</code>	File: file3.txt Size: 345 bytes Words: 50 Owner: <username>

→ Submission Guidelines:

- **Makefile:** Include a Makefile that compiles the program.
- **txt:** Provide instructions for compiling and running the program. Include any known issues or edge cases you handled.
- **Source Code:** Submit your source code and ensure it follows best coding practices, with appropriate comments and documentation.

case #1 ./hw03 /Labs/Lab06

main(argc=2, argv=[./hw03, Lab06])

⋮

else if (stat(argv[1], &st) == 0)

U = "0";

S_ISDIR(st.st_mode) ✓

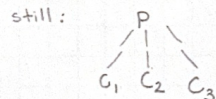
directory: /workspaces/CS532/Labs/Lab06

forking(argv[1], U, NULL);

-stdout Lab06
item → d-name {
a.txt
b.c
c.txt

• once parent has read every
item → d-name in dir
exits while(readdir(...)) +
executes

while(wait(&status) > 0) {
at this point tree
still:



as each child finishes ...



until ... (P)

wait() reports no more children,
parent closedir(dir) + returns
from forking()

• all 3 children
executing in parallel

dir = Lab06
while((item = readdir(dir)) != NULL)

filepath = /Lab06/item → d-name;

struct stat filestats;

if S_ISDIR(item → d-name) {
skip...
}

1) a.txt Parent (forking)

pid > 0
(back to while)

2) P (forking)

pid > 0
(back to while)

3) P₁ (forking)

pid > 0
...

C₀
pid = 0
args = {execmain, Lab06,
item → d-name, U,
usr, NULL}
execv(args[0], args);

pid = 0
args
execv(args[0], args);

pid = 0, args
execv(args[0], args);

→ execmain

```
int main(arg=6, argv=[0./execmain, 1Lab06, 2item→d_name, 3u, 4usr, 5NULL])
```

```
* yesfile = NULL
```

```
* correct = item→d_name (a.txt, b.c, c.txt)
```

```
dir = Lab06
```

```
while ((item = readdir(dir)) != NULL)
```

```
    if (item→d_name == correct)
```

```
        yesfile = item; → a.txt || b.c || c.txt  
        break;
```

```
    if (yesfile == NULL)
```

```
        stderr
```

```
        closedir(dir);
```

```
fullpath = /Lab06/item→d_name
```

```
if (u == "l") X → NO
```

```
    ;
```

```
else
```

```
    proceed = 1;
```

```
* ext = .txt || .c || .txt
```

```
if (proceed == 1) { ✓
```

```
    if (ext != NULL && ext == .txt)
```

```
        words(fullpath, &st, yesfile, u);
```

```
        ;
```

```
    else
```

File: b.c | Size: 682 bytes

* runs in
parallel for
all three
childs

runs for
a.txt
c.txt

words
 int words(/Lab06/item → d-name, txt, yesfile, u)

FILE *fptr = fopen(txt, "r");

while (getline(&buf, &size, fptr)) != -1) → reads file line by line
 ↓
 puts in buf...

int inword = 0;

for(i = 0; buf[i] != '\0'; i++)

if (isspace(buf[i]) → buf[5] in a.txt
 inword = 0; buf[4,7] in c.txt

else if (inword == 0) → enters at
 count++; buf[0,5] in a.txt
 inword = 1; buf[0,4,7] in c.txt
 a.txt count = 2
 c.txt count = 3

if (u == "1") x → NO
 ;

else

File: a.txt | Size: 12 bytes | Words: 2

File: c.txt | Size: 12 bytes | Words: 3

* DONE

-stdout

case # 2 ./hw03 -u bbelden /summer/me

me
a.txt
b.c
c.txt

main(argc=3, argv=[./hw03,⁰-u,¹bbelden,²/summer/me,³])

:

if (argv[1] == "-u") ✓

u = 1;

usr = bbelden

S_ISDIR(st.st_mode)

directory: /summer/me

forking(argv[3], u, usr)

:

same as case #1

in each execmain

same until/execmain /me item→d-name 0 1 2 3
u 4
usr

if (argv[3] = u | == 1) ✓

*owner = bbelden

if (owner != NULL + owner = usr)

proceed = 1;

else

proceed = 0;

:

if (proceed == 1)

if (ext != NULL ++ ext == .txt)

words(fullpath, +st, yesfile, argv[3]);

else if (argv[4] != NULL)

File: b.c | Size: 682 bytes |

Owner: bbelden

same as case #1

except:

:

if (u == "1")

File: a.txt | Size: 12 bytes |

Words: 2 | Owner: bbelden

: